

RAPPORT TECHNIQUE GLOBAL ET COMPLET : APPLICATION « CONCEPTION » — ÉDITEUR DE PLATEAUX

INTRODUCTION GÉNÉRALE

Ce document constitue le descriptif technique complet de l'**application Conception**, l'**éditeur de plateaux** des *Cartographes du Métro*. Elle permet de **créer**, **peindre**, **éditer** et **sauvegarder** des plateaux de jeu (au format `.txt`) qui seront ensuite jouables dans l'application Jeu.

Comme l'application Jeu, elle repose sur le patron **MVC (Modèle-Vue-Contrôleur)** avec **Java Swing/AWT**, et sépare strictement la **logique métier** (plateau, graphe, lecture/écriture de fichiers, validations) de l'**IHM**. Le **Contrôleur** est l'unique passerelle entre les deux.

Le code est organisé en trois paquets : `plateau` (le Contrôleur), `plateau.ihm` (les fenêtres et panneaux) et `plateau.metier` (le cœur logique autonome).

Le **flux d'utilisation** est séquentiel : **Accueil** -> **Configuration** (saisie des paramètres : dimensions, joueurs, stations, arrondissements, taille des cases) -> **Création** (peinture des arrondissements à la souris) -> **Jeu / Aperçu** (placement des stations et des départs, import d'un plateau existant, sauvegarde).

1. LE PACKAGE PRINCIPAL (`plateau`)

`Contrôleur.java`

Classe pivot. Elle détient le modèle courant et la configuration de la partie en cours d'édition, et expose au reste de l'application des méthodes de lecture/écriture contrôlées.

Attributs

- `ihm (FrameAccueil)` : fenêtre d'accueil instanciée au lancement.
- `metier (Plateau)` : plateau en cours d'édition.
- `nbJoueurs (int)` : nombre de joueurs configuré.
- `nbStations (int)` : nombre de types de stations configuré.

Descriptif granulaire des méthodes

- `Contrôleur()` (*constructeur*) : crée un `Plateau` 10×10 par défaut puis affiche la `FrameAccueil`.
- `initialiserPlateau(int largeur, int hauteur)` : écrase l'ancien modèle pour recréer un `Plateau` vierge aux dimensions demandées.
- `setConfigJeu(int nbJoueurs, int nbStations)` : enregistre la configuration de la partie.
- `getNbJoueurs()` / `getNbStations()` : accesseurs de configuration.
- `chargerPlateau(File fichier)` : délègue la lecture au `ChargeurPlateau` ; remplace le modèle si la lecture réussit.
- `getLargeur()`, `getHauteur()`, `getArrondissement(int)`, `getStation(int)`, `getDepart(int)` : accesseurs en lecture sur le plateau.

- `affecterStation(int, int)`, `affecterDepart(int, int)`, `affecterArrondissement(int, int)` : mutateurs déposant une station, un départ ou un arrondissement sur une case.
- `aArete(int i, int j)` : interroge le graphe sur l'existence d'une liaison.
- `getNbAretes()` : renvoie le nombre total d'arêtes du graphe.
- `genererAretesAuto()` : commande au graphe de recalculer automatiquement toutes ses liaisons à partir des stations posées.
- `enregistrerPlateau(String nomFichier)` : délègue l'écriture du fichier à l'`EnregistreurPlateau`.
- `getImageFond()` / `getImageFond2()` : résolvent les chemins absolus des images de fond.
- `mettreAJourConfigurationDepuisPlateau()` : recalcule nbJoueurs et nbStations à partir du contenu réel d'un plateau chargé (via `ConfigurationPlateau`).
- `validerDepartsPlateau()` : vérifie, via `ValideurPlateau`, que chaque joueur possède exactement un départ.
- `main(String[] a)` : point d'entrée ; instancie un `Controlleur`.

2. LE PACKAGE INTERFACE GRAPHIQUE (`plateau.ihm`)

`FrameAccueil.java`

Fenêtre d'accueil (724×965) qui héberge le `PanelAccueil`, fixe la fermeture (`EXIT_ON_CLOSE`), se centre et s'affiche.

`PanelAccueil.java`

Panneau d'accueil avec image de fond et deux boutons. - `PanelAccueil(...)` (*constructeur*) : charge l'image de fond, crée les boutons « **Configuration du plateau** » et « **Règles** », les centre dans un `GridBagLayout`, et branche les écouteurs. - `actionPerformed(...)` : « Configuration » ouvre la `FrameConfiguration` et masque l'accueil ; « Règles » ouvre le PDF des règles via `Desktop.open`. - `paintComponent(...)` : étire l'image de fond sur tout le panneau.

`FrameConfiguration.java`

Fenêtre (`FlowLayout`, `pack()`) hébergeant le formulaire de configuration.

`PanelConfiguration.java`

Formulaire de saisie des paramètres d'un nouveau plateau. - **Attributs** : six labels et six champs de saisie **statiques et publics** (`txtLargeur`, `txtHauteur`, `txtJoueurs`, `txtStation`, `txtArrondissements`, `txtTailleCases`) — déclarés `static public` afin d'être lus directement par le panneau de création — plus les boutons `btnValider`/`btnAnnuler`. - `PanelConfiguration(...)` (*constructeur*) : dispose les couples label/champ dans une grille `GridLayout(7, 2)` (chaque champ étant enveloppé par `creerPanelAjuste` pour éviter qu'il ne s'étire), sur un fond translucide. - `creerPanelAjuste(JTextField)` : utilitaire encapsulant un champ dans un `FlowLayout` gauche pour contrôler sa largeur. - `paintComponent(...)` : dessine l'image de fond puis un voile noir translucide. - `actionPerformed(...)` : sur « Valider », vérifie que tous les champs sont remplis, convertit les valeurs et applique une **batterie de tests de cohérence** (voir encart) ; si tout est valide, initialise le plateau et la configuration puis ouvre la `FrameCreation`. Sur « Annuler », vide instantanément tous les champs.

Analyse : validation des paramètres de configuration. Avant de créer le plateau, le formulaire refuse (avec message console) : tout champ vide, toute valeur non numérique (NumberFormatException), plus de **6 types de stations**, plus de **4 joueurs**, plus de **20 arrondissements**, toute valeur **négative ou nulle**, et une largeur ou hauteur **inférieure au nombre de joueurs** (incohérence géométrique). Aucun modèle n'est créé tant qu'un test échoue.

FrameCreation.java

Fenêtre (FlowLayout, pack()) hébergeant le panneau de peinture du plateau.

PanelCreation.java

Écran de **peinture des arrondissements** : une grille de cases vierges que l'on colorie à la souris. -

Attributs : lit ses dimensions directement dans les champs statiques de PanelConfiguration ; tabCases[] (les cellules JPanel), btnArrondissements[] (un bouton coloré par arrondissement), btnValider/btnRetourConfiguration/btnPasserAuJeu, txtNomFichier, arrondissementSelectionne, couleurSelectionnee, plus les tableaux de noms (1er...20ème) et de couleurs. - **PanelCreation(...)** (constructeur) : construit la grille de cases (GridLayout), la colonne des boutons d'arrondissements colorés à gauche, et la barre du bas (champ de nom + boutons), puis branche les écouteurs de souris et d'action. - **colorierCaseSousSouris(MouseEvent)** : identifie la case sous le curseur (getComponentAt), la colorie avec la couleur sélectionnée et **affecte l'arrondissement** correspondant dans le modèle. - **mousePressed / mouseDragged** : appellent la peinture — le **glisser** (drag) permet de colorier plusieurs cases d'un coup, comme un pinceau. - **actionPerformed(...)** : un clic sur un bouton d'arrondissement sélectionne sa couleur et son numéro ; « Valider » vérifie que **toutes les cases sont peintes** et que le **nom n'est pas vide**, puis enregistre le plateau et ouvre l'écran de jeu ; « Retour » revient à la configuration ; « Importation Uniquement » ouvre directement l'écran de jeu.

Analyse : peinture par glisser (drag-paint). En implémentant à la fois MouseListener et MouseMotionListener, le panneau réagit au **maintien-déplacement** de la souris : mouseDragged colorie en continu chaque case survolée. C'est ce qui donne la sensation d'un pinceau et permet de remplir rapidement de grandes zones d'un même arrondissement.

FrameJeu.java

Fenêtre hébergeant le PanelJeu (l'éditeur/aperçu), centrée et emballée (pack()).

PanelJeu.java

Écran le plus dense : il combine un **panneau d'outils** à gauche et un **aperçu interactif du plateau** à droite, avec tracé des arêtes. - **Attributs** : colonne gauche (labels d'info joueurs/stations/arêtes, combo des fichiers, boutons import/création, radios de mode rbStation/rbDepart et combos associées, bouton de sauvegarde) ; à droite panelApercuGrille (la grille) ; cache stationImages[] ; un ListenerJeu centralisé. - **PanelJeu(...)** (constructeur) : assemble la colonne d'outils (BoxLayout vertical) et le panneau d'aperçu. Ce dernier est un JPanel **anonyme** dont la méthode paint est redéfinie pour **tracer les arêtes du graphe**. Les radios « Station »/« Départ » activent/désactivent leur combo respective. - **mettreAJourComboFichiers()** : liste les fichiers .txt du dossier sauvegarde/ dans la combo d'import. - **mettreAJourComboPlacement()** : remplit dynamiquement les combos de stations (selon nbStations) et de départs (selon nbJoueurs). - **chargerApercu(File)** : charge un plateau, met à jour la configuration et les labels (joueurs, stations, **arêtes**), reconstruit la grille en instanciant un CasePanel par case, et réajuste la fenêtre. - **caseCliquee(int index)** : selon le mode actif (station ou départ) et la sélection, délègue le placement à GestionnairePlacement, **recalcule automatiquement les arêtes** et repeint l'aperçu. -

`getImageStation(int)` : cache paresseux des images de stations. - Plus les getters exposés au `ListenerJeu`.

Géométrie : tracé des arêtes dans l'aperçu (paint anonyme). Pour relier deux stations, le code récupère les deux `CasePanel` concernés via `getComponent(i)/getComponent(j)`, puis calcule le **centre** de chacun en pixels : $x = \text{comp.getX()} + \text{comp.getWidth()}/2$ (idem pour y). Ce **barycentre** permet de tracer des lignes nettes et antialiasées (`RenderingHints`, `BasicStroke(3)`) qui partent bien du milieu des cases, et non de leur coin.

`CasePanel.java`

Représente **une case** de l'aperçu et gère son rendu et son clic. - `CasePanel(index, ctrl, panelJeu)` (*constructeur*) : fixe une taille et une bordure, et branche un `MouseListener` qui prévient le `PanelJeu` du clic sur cette case. - `paintComponent(Graphics g)` : dessine en trois temps — **(1)** le fond coloré selon l'arrondissement, **(2)** l'**image de la station** si présente, **(3)** le texte du **départ** (ex. « D1 ») si la case est un point de départ.

`ListenerJeu.java`

Écouteur centralisé des boutons du `PanelJeu` (héritant `ActionListener`). - `actionPerformed(...)` : « Importer » charge l'aperçu du fichier sélectionné ; « Créer Nouveau » rouvre la configuration et ferme la fenêtre courante ; « Sauvegarder » **valide d'abord les départs** (`validerDepartsPlateau`), puis enregistre le plateau et ferme la fenêtre.

3. LE PACKAGE LOGIQUE MÉTIER (`plateau.metier`)

Paquet **autonome**, sans dépendance graphique.

`Plateau.java`

Modèle maître : dimensions, trois tableaux d'entiers (`tabArrondissements`, `tabStations`, `tabDeparts`) indexés par case, et le graphe. Expose les accesseurs/mutateurs bornés correspondants et `getGraphe`.

`GraphePlateau.java`

Matrice d'adjacence booléenne du réseau de stations. - `aArete(i, j)` : existence d'une liaison. - `ajouterArete(i, j)` : ajout **symétrique** (graphe non orienté). - `vider()` : réinitialise la matrice. - `aDesAretes()` : indique si au moins une arête existe. - `getNbAretes()` : compte les arêtes (en ne parcourant que le triangle supérieur $j > i$ pour ne pas compter deux fois). - `genererAretesAuto(Plateau)` : génération automatique des liaisons (voir encart).

Analyse spatiale : génération des arêtes (genererAretesAuto). Algorithme par **lancer de rayon** (*ray-casting*). Pour chaque station, il explore 4 directions (Est, Sud-Est, Sud, Sud-Ouest — les 4 opposées étant obtenues par symétrie) en **avançant case par case** jusqu'à la **première** station rencontrée, à laquelle il se relie avant de s'arrêter. Deux stations sont voisines si elles sont alignées et qu'**aucune autre station ne s'intercale**.

`ChargeurPlateau.java`

Décodeur lisant un fichier `.txt` (séparateur `;`) pour instancier un `Plateau` : en-tête `largeur;hauteur`, lignes de cases `index;arrondissement;station;depart`, puis lignes d'arêtes `source;destination`.

Analyse : robustesse de la lecture (charger). Renvoie `null` au moindre défaut : fichier absent/illisible, en-tête != 2 valeurs, dimensions hors bornes (≤ 0 ou > 100), indices non séquentiels, valeurs hors plage (arrondissement 0–20, station 0–10, départ 0–4), nombre de cases incorrect, valeurs non numériques. À défaut d'arêtes dans le fichier, elles sont **générées automatiquement**.

EnregistreurPlateau.java

Sauvegarde un plateau au format `.txt`. - **enregistrer(Plateau, nomFichier)** : régénère d'abord les arêtes, crée le dossier `sauvegarde/` si besoin, **ajoute l'extension `.txt`** si absente, puis écrit l'en-tête de dimensions, une ligne par case (`index;arrondissement;station;depart`), et une ligne par arête (`i;j`, triangle supérieur uniquement). Renvoie `true` en cas de succès.

Analyse : symétrie lecture/écriture. L'EnregistreurPlateau produit **exactement** le format attendu par le ChargeurPlateau (même ordre, mêmes séparateurs), garantissant un cycle **sauvegarde -> rechargement** sans perte. Les arêtes sont écrites une seule fois ($j > i$) puisque le graphe est non orienté.

ConfigurationPlateau.java

- **detecterMaxJoueursEtStations(Plateau, nbJoueursDefault, nbStationsDefault)** : parcourt le plateau pour déduire le **nombre réel** de joueurs (max des départs) et de types de stations (max des stations), utilisé après le chargement d'un plateau existant.

ValideurPlateau.java

- **validerDeparts(Plateau, nbJoueurs)** : vérifie que **chaque joueur possède exactement UN départ** sur le plateau ; sinon, affiche un message d'erreur précis (nombre de départs trouvés) et renvoie `false`. C'est la garde finale avant qu'un plateau soit déclaré valide.

GestionnairePlacement.java

- **gererClic(index, modeSelection, selectionItem, ctrl)** : applique le placement selon le mode. En mode **STATION**, traduit le nom choisi en numéro de type (ou 0 pour « Aucun ») et l'affecte à la case. En mode **DEPART**, garantit l'**unicité** : il **efface d'abord** l'éventuel départ existant de ce joueur ailleurs sur le plateau, puis pose le nouveau départ sur la case cliquée.

UtilitaireJeu.java

Boîte à outils statique : `getNomsStations()` (noms thématiques), `getCouleurs()` (20 couleurs d'arrondissements), `getCheminImageStation(numero)` (résolution du chemin d'image dans plusieurs emplacements).